Clayton Cooper

CS-370

6/19/2026

Pirate Intelligent Agent: Design Defense

Introduction

The design of the pirate intelligent agent created for the treasure hunt game detailed in CS 370 Project Two is defended in this essay. The agent is an example of deep Q-learning, a type of reinforcement learning, used to solve a maze-based pathfinding issue. The following sections examine how a human and a machine navigate the same maze, evaluate the intelligent agent's role in pathfinding, and assess the deep Q-learning algorithm that trained it.

Human Versus Machine Approaches to Problem Solving

In order to solve the maze, a human would probably rely on vision and spatial reasoning. They could look at the entire 8x8 grid at once, mentally trace open paths from the starting corner to the treasure in the opposite corner, and rule out dead ends almost instantly. If they come across a wall or a previously visited cell, they would go back and try a different route, relying on memory of where they have already been and general intuition about which direction moves them closer to the goal. This is a largely deliberate, holistic process: the human creates an internal mental map of the maze before making many physical moves.

The pirate agent approaches the same maze in a very different way. It only sees a flattened numerical representation of the environment, known as the envstate, at each step; it lacks an intrinsic sense of geometry or a comprehensive understanding of the maze (Russell & Norvig, 2021). Based on either random exploration or the Q-values predicted by its neural network, the agent choose a left, right, up, or down action for each state. The environment returns a reward

and a new state following each move, and the agent uses the GameExperience class to store that transition in memory. The network's weights are modified over thousands of repeated episodes so that the predicted Q-values progressively show which actions result in penalties and which lead to the treasure. The agent eventually approximates an action-value function by trial and error and statistical reinforcement; it has no mental understanding of the maze.

The fundamental distinction between the two methods is how the underlying "knowledge" is represented and acquired. The machine has no abstraction at all; it must repeatedly experience the maze, accumulate reward signals, and gradually encode useful patterns into numerical network weights before it can reliably reach the goal. In contrast, humans reason abstractly and can solve the maze in a single attempt by visual inspection (Sutton & Barto, 2018).

The Purpose of the Intelligent Agent in Pathfinding

The agent's goal is to learn a policy that associates maze situations with behaviors that consistently lead to the prize without being explicitly provided with a path. This necessitates striking a balance between two opposing behaviors: exploration and exploitation. Exploitation means the agent takes the action it now believes is best, based on the highest anticipated Q-value from its neural network. Exploration means the agent instead takes a random valid action, even if the network does not currently evaluate it as optimal, in order to explore states and rewards it has not yet experienced (Sutton & Barto, 2018).

Early in training, the agent's Q-value predictions are essentially noise because its network has not yet learnt anything significant. The agent would never figure out the actual arrangement of rewards in the maze if it was abused from the very first epoch. Instead, it would just repeat whatever random action the untrained network occurred to favor. since of this, the agent investigates nearly every action at the start of training since the epsilon parameter, which

regulates the likelihood of exploring versus exploiting, starts high (epsilon = 1.0 in this implementation). Epsilon decreases toward a tiny minimum value (0.05 in our version) as training goes on and the network's predictions get more accurate. As a result, the agent gradually makes use of what it has learnt while still periodically exploring to avoid becoming trapped in a less-than-ideal path. The ideal ratio for this problem is a decaying epsilon schedule, which maintains a small, fixed exploration rate even after the win rate is high. This is because the maze is static, and once a model finds numerous winning paths, it gains more from honing and validating those paths than from continuing to explore at a high rate.

Because the treasure hunt can be naturally framed as a Markov decision process—a set of states (pirate positions), actions (four possible moves), and rewards (positive for reaching the treasure, negative for invalid moves, revisits, or running out of moves)—reinforcement learning is well suited to this type of pathfinding problem (Russell & Norvig, 2021). The agent gradually determines which action sequences, from any beginning cell, result in the maximum cumulative reward by repeatedly executing episodes and updating a value function based on observed rewards. Finding the most effective move sequence from a start state to a target state is the exact objective of pathfinding, but it is learnt via experience rather than calculated analytically like in a graph-search algorithm like A*.

Evaluating the Use of Algorithms to Solve Complex Problems

The algorithm used by the pirate agent is an implementation of deep Q-learning. The amount of potential pirate positions and maze configurations makes it hard to scale a lookup table, but classic Q-learning keeps a table of Q-values for each possible state-action pair. By substituting a neural network that receives a state as input and produces an estimated Q-value for each of the four possible actions, deep Q-learning overcomes this problem (Mnih et al., 2015). The network

constructed in build_model() in this implementation is a tiny, fully connected (Dense) network with two hidden layers utilizing PReLU activations, followed by an output layer with one neuron per action that is compiled using mean-squared-error loss and the Adam optimizer.

The deep Q-learning loop is used to train the network. The pirate is positioned at a randomly chosen free cell for each epoch, and the episode continues until the pirate either wins, loses, or uses up all of its move budget. The agent selects whether to explore or exploit at each stage, uses the act() method of the TreasureMaze environment to take the action that results, and then saves the transition (prior state, action, reward, new state, game status) in a GameExperience replay buffer. Training targets are then calculated by randomly sampling a batch of these stored transitions. For terminal transitions, the target is simply the observed reward; for non-terminal transitions, the target is the observed reward plus a discounted estimate of the best future Q-value, predicted by a different target network that is periodically synchronized with the main model. Since updating the main network with its own constantly changing predictions as a frequent target would make learning extremely unstable, this target network and the experience replay buffer stabilize training (Mnih et al., 2015).

The implementation monitors a rolling win rate following each epoch. The agent is gradually moved from exploration to exploitation using the fading epsilon schedule previously mentioned. The completion_check() function confirms that the trained model can indeed reach the treasure starting from any valid free cell in the maze, not simply the cells that happened to be sampled most recently, once the win rate continuously surpasses the selected threshold. This last verification step is crucial since an agent could seem to be performing well over a brief rolling window just by coincidence. The completion_check() method verifies that this is not a lucky run but rather true, generalized learning.

Because deep Q-learning scales to bigger or more complicated state spaces than tabular Q-learning and because the combination of experience replay and a target network results in a stable, steadily developing policy, deep Q-learning proved to be an excellent algorithmic solution for this challenge overall. Instead of memorizing a single trajectory, the resulting pirate agent may locate a path to the loot from any legitimate starting place in the maze, proving that the agent has acquired a truly effective policy.

Conclusion

Constructing the pirate intelligent agent demonstrates a fundamental concept in artificial intelligence: rather than explicitly hand-coded rules, complicated behavior, like navigating a maze, can arise from a relatively simple learning algorithm interacting with its surroundings on a regular basis. The pirate agent uses repetitive trial and error under the guidance of reinforcement signals to progressively encode a good policy into the weights of a neural network, whereas a human uses visual reasoning and an internal mental map to solve the maze. The key design choices that made this technique dependable were striking a balance between exploration and exploitation and stabilizing learning via experience replay and a target network.

References

Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A. A., Veness, J., Bellemare, M. G., Graves, A., Riedmiller, M., Fidjeland, A. K., Ostrovski, G., Petersen, S., Beattie, C., Sadik, A., Antonoglou, I., King, H., Kumaran, D., Wierstra, D., Legg, S., & Hassabis, D. (2015). Human-level control through deep reinforcement learning. Nature, 518(7540), 529–533. https://doi.org/10.1038/nature14236

Russell, S., & Norvig, P. (2021). Artificial intelligence: A modern approach (4th ed.). Pearson.

Sutton, R. S., & Barto, A. G. (2018). Reinforcement learning: An introduction (2nd ed.). MIT Press.